

AfterReload — Documentación de Fin de Curso

Ciclo Formativo: Desarrollo de Aplicaciones Web (DAW)

*Proyecto: AfterReload — Plataforma web de matchmaking para Counter-Strike

*Tecnología principal: Laravel 12 + PHP 8.2

1. Introducción

1.1 Descripción del proyecto

AfterReload es una plataforma web de matchmaking y gestión de servidores diseñada para comunidades de Counter-Strike. Su propósito es centralizar en un único sistema la autenticación de jugadores mediante Steam, la organización de partidas a través de lobbies, el control remoto de servidores de juego y el registro de estadísticas y progresión de los jugadores.

1.2 Contexto y motivación

Las comunidades de Counter-Strike autogestionadas dependen habitualmente de herramientas externas y dispersas para organizar partidas: grupos de Discord para coordinar jugadores, paneles de Pterodactyl para gestionar servidores, hojas de cálculo para estadísticas, y bots para gestión de acceso. Esta dispersión genera fricciones, errores de coordinación y una experiencia de usuario degradada.

AfterReload surge como respuesta a esta problemática. El objetivo es ofrecer una plataforma propia que integre todos estos procesos en un entorno web cohesionado, accesible desde el navegador y sin dependencias de herramientas de terceros para las operaciones cotidianas.

1.3 Alcance del proyecto

El sistema cubre los siguientes ámbitos funcionales:

- Autenticación de usuarios mediante la identidad de Steam (OAuth 2.0).
- Gestión de servidores de juego diferenciando entre servidores públicos y matchmaking.
- Sistema de lobbies con selección de equipo, indicador de estado de preparación y sincronización en tiempo real.
- Integración con el plugin Get5 para la orquestación de partidas competitivas.
- Control remoto de servidores mediante el protocolo RCON.
- Procesamiento automático de resultados en servidores públicos a partir de logs del motor de juego.
- Sistema de puntos y créditos como mecanismo de progresión del jugador.
- Panel de administración centralizado para la gestión de usuarios, roles, baneos y servidores.
- Interfaz multiidioma con soporte para español e inglés.

1.4 Tecnologías empleadas

| Capa | Tecnología | Versión |

#####

| Backend | Laravel (PHP) | 12.0 / PHP 8.2+ |

| ORM | Eloquent | Laravel 12 |

| Frontend | Blade + Tailwind CSS | Tailwind 4.0 |

| Build | Vite | 7.0.7 |

| HTTP client | Axios | 1.11.0 |

| Base de datos | SQLite / MySQL | - |

| Autenticación | Steam OAuth (Socialite) | ^4.3 |

| Testing | PHPUnit | 11.5.3 |

| Servidor web | Nginx + PHP-FPM | 8.2 |

| Control de versiones | Git | - |

2. Planificación

2.1 Metodología de trabajo

El proyecto se desarrolló de forma iterativa e incremental, sin aplicar una metodología formal de gestión de proyectos como Scrum o Kanban, dado el carácter individual del desarrollo. Sin embargo, sí se siguió una progresión estructurada en fases con hitos claros, priorizando la funcionalidad sobre la completitud.

Cada fase se inició una vez que la anterior producía un resultado estable y verificable. Esto permitió detectar problemas de integración de forma temprana y ajustar el alcance sin comprometer la integridad del sistema.

2.2 Fases del proyecto

Fase 1 — Base del sistema (semanas 1–2)

Objetivos:

- Creación del proyecto Laravel desde cero.
- Definición del esquema inicial de base de datos con sus migraciones.
- Configuración de autenticación mediante Steam (Socialite).
- Estructura de rutas, controladores y vistas base con Blade.
- Layout principal con sidebar, topbar y sistema de navegación.

Resultado: Sistema funcional con login via Steam, gestión de sesiones y estructura MVC completa.

Fase 2 — Gestión de servidores y panel admin (semanas 3–4)

Objetivos:

- Modelo `Server` con campos para IP, puerto, tipo y estado.
- Panel de administración para crear, editar y eliminar servidores.
- Vista de listado de servidores con actualización automática.

- Sistema de roles de usuario (user, admin, betatester, store).
- Middleware de control de acceso y gestión de usuarios baneados.

Resultado: Panel de administración operativo con CRUD de servidores y control de usuarios.

Fase 3 — Lobbies y matchmaking (semanas 5–6)

Objetivos:

- Modelo `Lobby` y tabla pivote `lobby\_user` para la relación N:M entre usuarios y lobbies.
- Controlador de lobby con lógica de entrada, salida, selección de equipo y estado de preparación.
- Sistema de poda de usuarios inactivos mediante TTL de presencia.
- Integración con Get5 para construcción de configuraciones de partida.
- Webhook de eventos Get5 para procesamiento de resultados competitivos.

Resultado: Flujo completo de matchmaking desde el lobby hasta la conclusión del match y aplicación de puntos.

Fase 4 — Servidores públicos y RCON (semanas 7–8)

Objetivos:

- Implementación del cliente RCON para comunicación con servidores de juego.
- Servicio de procesamiento de logs de servidores públicos.
- Detección automática de finales de partida y aplicación de recompensas.
- Servicio de verificación del estado de runtime del servidor local (LGSM).

Resultado: Los servidores públicos pueden detectar y registrar partidas finalizadas sin intervención manual.

Fase 5 — Internacionalización, pruebas y documentación (semanas 9–10)

Objetivos:

- Traducción completa de la interfaz al español e inglés (142 claves).
- Pruebas automatizadas con PHPUnit para el procesador de logs.
- Documentación técnica y de instalación.
- Revisión general del sistema, limpieza de código y ajustes finales.

Resultado: Sistema completo, documentado y con cobertura de pruebas en los módulos críticos.

2.3 Recursos empleados

- **Hardware**: Servidor Linux con Ubuntu 24.04 LTS para el despliegue.
- **Software de desarrollo**: Editor de código, terminal, navegador para pruebas.
- **Servicios externos**: Cuenta de desarrollador de Steam para OAuth, servidor CS:GO con Get5 instalado.
- **Control de versiones**: Repositorio Git local con commits progresivos.

3. Análisis

3.1 Identificación del problema

Las comunidades de Counter-Strike que gestionan sus propios servidores se enfrentan a varios problemas recurrentes:

1. **Fragmentación de herramientas**: La coordinación de jugadores, el control de servidores y el registro de estadísticas se realiza en plataformas distintas y desconectadas.
2. **Gestión manual de lobbies**: La organización de partidas depende de comunicación en canales de texto, con alta probabilidad de errores humanos.
3. **Ausencia de progresión propia**: Los servidores públicos no ofrecen un sistema de puntos vinculado a la identidad del jugador dentro de la comunidad.
4. **Control de acceso rudimentario**: Sin una plataforma propia, es difícil aplicar restricciones (baneos, roles) de forma coherente.
5. **Counter-Strike Legacy (Global Offensive) está obsoleto** lo cual dificulta encontrar servidores enfocados a un ámbito competitivo.

3.2 Requisitos funcionales

****RF-01 — Autenticación mediante Steam****

El sistema debe permitir que los usuarios inicien sesión utilizando su cuenta de Steam. No se requiere registro manual ni contraseña propia.

****RF-02 — Gestión de servidores****

Los administradores deben poder crear, editar y eliminar servidores. Cada servidor tiene nombre, IP, puerto, tipo (público o matchmaking) y estado (online/offline).

****RF-03 — Vista de servidores para usuarios****

Los usuarios autenticados deben ver la lista de servidores disponibles con el número de jugadores actual y su estado.

****RF-04 — Lobbies para servidores matchmaking****

Cada servidor de tipo matchmaking tiene un lobby asociado. Los usuarios pueden unirse, seleccionar equipo (CT o T) y marcar su estado de preparación.

****RF-05 — Inicio automático de partida****

Cuando todos los jugadores requeridos estén listos y los equipos estén equilibrados, el sistema debe iniciar la partida en el servidor a través del protocolo RCON y la integración con Get5.

****RF-06 — Procesamiento de resultados****

Al finalizar una partida (competitiva o pública), el sistema debe registrar el resultado y aplicar puntos y créditos a los jugadores participantes según si ganaron o perdieron.

****RF-07 — Ranking público****

El sistema debe mostrar una tabla con los 50 jugadores con mayor puntuación.

****RF-08 — Panel de administración****

Los administradores deben disponer de un panel centralizado para gestionar usuarios (cambiar rol, banear, modificar puntos y créditos) y servidores.

****RF-09 — Internacionalización****

La interfaz debe estar disponible en español e inglés, con posibilidad de cambiar el idioma desde la barra de navegación.

****RF-10 — Gestión de usuarios baneados****

Los usuarios baneados deben ser redirigidos a una página informativa y no deben poder acceder a las funcionalidades del sistema.

3.3 Requisitos no funcionales

****RNF-01 — Rendimiento****: La vista de servidores debe actualizarse de forma automática sin recargar la página completa. El intervalo de actualización es de 5 segundos.

****RNF-02 — Seguridad****: La autenticación se delega a Steam mediante OAuth. Las rutas de administración están protegidas por middleware de verificación de rol. Los comandos RCON se envían desde el servidor, nunca desde el cliente.

****RNF-03 — Consistencia de datos****: El sistema debe evitar que un mismo evento de partida sea procesado más de una vez, utilizando hashes SHA-256 como identificadores únicos de eventos.

****RNF-04 — Mantenibilidad****: El código sigue el patrón MVC de Laravel. La lógica de negocio compleja se encapsula en servicios independientes, no en controladores.

****RNF-05 — Escalabilidad****: La base de datos puede configurarse como SQLite (entornos de desarrollo o instalaciones pequeñas) o MySQL/MariaDB (producción con mayor carga).

****RNF-06 — Despliegue****: El sistema debe poder instalarse en un servidor Linux estándar con Nginx, PHP-FPM y Composer, sin dependencias propietarias.

3.4 Actores del sistema

Actor	Descripción
-------	-------------

#####

Usuario no autenticado	Puede ver la página de bienvenida y el ranking. No puede acceder a servidores ni lobbies.
------------------------	---

Usuario autenticado	Accede a servidores, lobbies, ranking y su perfil.
---------------------	--

Beta tester	Usuario con acceso anticipado a funcionalidades en desarrollo.
-------------	--

Acceso tienda	Usuario con permiso para acceder al módulo de tienda e inventario.
---------------	--

Administrador	Acceso completo, incluyendo panel de administración.
---------------	--

Sistema Get5	Actor externo que envía eventos de partido al webhook de la API.
--------------	--

Servidor CS:GO	Actor externo que genera logs de partidas públicas.
----------------	---

3.5 Restricciones técnicas detectadas

- Los servidores de CS:GO Legacy requieren librerías de 32 bits (`lib32gcc-s1`, `lib32stdc++6`), que deben instalarse en el sistema operativo independientemente de la aplicación web.
- La integración con Get5 requiere que el servidor de juego tenga el plugin instalado y configurado con la URL del webhook.
- El estado de presencia en lobbies tiene un TTL de 20 segundos. Si un usuario cierra el navegador sin salir explícitamente, el sistema lo expulsa automáticamente en el siguiente ciclo de poda.
- La base de datos del servidor de juego (para sincronización de niveles con SourceMod) es una conexión separada de la base de datos principal de la aplicación.

4. Implementación

4.1 Estructura de directorios

afterreload/

```
|— app/
|   |— Events/
|   |   └─ LobbyUpdated.php    # Evento broadcastable
|   |— Http/
|   |   └─ Controllers/
|   |       └─ Admin/
|   |           └─ AdminController.php
|   |           └─ Api/
|   |               └─ Get5Controller.php
|   |               └─ Auth/
|   |                   └─ SteamController.php
|   |                   └─ Lobby/
|   |                       └─ LobbyController.php
|   |                       └─ LanguageController.php
|   |                       └─ RankingController.php
|   |                       └─ ServerController.php
|   |   └─ Middleware/
|   |       └─ EnsureNotBanned.php
|   |       └─ SetLocale.php
|   └─ Models/
|       └─ Lobby.php
|       └─ LobbyMatch.php
```

- | | | └─ MatchResult.php
- | | | └─ Server.php
- | | └─ User.php
- | └─ Services/
 - | └─ LocalCsgoserverRuntimeService.php
 - | └─ PublicMatchResultProcessor.php
 - | └─ RconClient.php
- └─ database/
 - | └─ migrations/ # 10 migraciones versionadas
- └─ lang/
 - | └─ en.json # 142 claves en inglés
 - | └─ es.json # 142 claves en español
- └─ resources/
 - | └─ views/
 - | └─ layouts/
 - | | └─ app.blade.php # Layout principal
 - | └─ partials/
 - | | └─ sidebar.blade.php
 - | | └─ topbar.blade.php
 - | └─ admin/
 - | | └─ index.blade.php
 - | └─ welcome.blade.php
 - | └─ home.blade.php
 - | └─ lobby.blade.php
 - | └─ ranking.blade.php
 - | └─ ...

```
|— routes/
|  |— api.php
|  |— web.php
|— tests/
|  |— Unit/
|     |— PublicMatchResultProcessorTest.php
|— composer.json
|— package.json
|— vite.config.js
```

4.2 Módulo de autenticación con Steam

La autenticación implementa el flujo OAuth 2.0 a través del paquete `socialiteproviders/steam`. El proceso es el siguiente:

1. El usuario hace clic en "Iniciar sesión con Steam".
2. `SteamController::redirectToSteam()` redirige al servidor de autenticación de Steam.
3. Steam autentica al usuario y redirige de vuelta al callback de la aplicación.
4. `SteamController::handleSteamCallback()` recibe el token y crea o actualiza el registro del usuario en la base de datos con los datos del perfil de Steam (nickname, avatar, SteamID64).
5. La sesión se inicia y el usuario es redirigido a la vista de servidores.

La verificación de si un usuario es administrador se realiza de dos formas complementarias: mediante el campo `role` en la base de datos o mediante la lista `ADMIN\_STEAM\_IDS` en las variables de entorno. Esto permite establecer administradores sin necesidad de acceder a la base de datos directamente.

4.3 Módulo de lobbies

El `LobbyController` es el componente más complejo del sistema. Sus responsabilidades incluyen:

****Poda de usuarios inactivos**:** Los registros en la tabla pivote ``lobby_user`` tienen un timestamp de actualización. Si un registro no se ha actualizado en los últimos 20 segundos, se considera inactivo y se elimina. Esto resuelve el problema de los "jugadores fantasma" que cerraron el navegador sin salir del lobby.

****Verificación de equipos**:** Antes de iniciar un match, el sistema verifica que ambos equipos tengan el mismo número de jugadores y que todos estén marcados como listos.

****Sincronización de estado**:** El endpoint ``/lobby/{server}/status`` devuelve un JSON con el estado actual del lobby. La vista de lobby hace polling a este endpoint cada 3 segundos para mostrar actualizaciones en tiempo real sin necesidad de WebSockets.

****Manejo de presencia TTL**:** Cada vez que el cliente hace una petición al endpoint de status, el timestamp del registro ``lobby_user`` se actualiza. Si el cliente deja de hacer peticiones (por cierre del navegador), el registro expira y el usuario es expulsado en el siguiente ciclo.

4.4 Módulo de integración con Get5

El ``Get5Controller`` gestiona dos puntos de entrada:

****Endpoint ``GET /api/get5/match/{lobby}``**:** Cuando el servidor de juego necesita cargar la configuración del match, consulta este endpoint. El controlador construye y devuelve un JSON con la estructura que Get5 espera, incluyendo los SteamIDs de cada equipo, el mapa, el nombre del match y la URL del webhook de eventos.

****Endpoint ``POST /api/get5/events``**:** Get5 envía a este endpoint todos los eventos del match: inicio de mapa, fin de mapa, resultado final, etc. El controlador:

1. Valida el token de autenticación en la cabecera.
2. Computa el hash SHA-256 del payload para detectar duplicados.
3. Procesa el evento según su tipo (``series_result``, ``map_result``, etc.).
4. Identifica al equipo ganador y al perdedor.

5. Aplica puntos y créditos a los jugadores de cada equipo.
6. Marca el lobby y el match como completados.
7. Envía comandos RCON al servidor para restablecer su configuración.

4.5 Servicio RCON

`RconClient` implementa el protocolo RCON de Valve directamente en PHP, sin dependencias externas. El protocolo funciona sobre TCP y utiliza un formato de paquete binario específico:

[longitud: 4 bytes][id: 4 bytes][tipo: 4 bytes][datos: variable][terminador: 2 bytes]

El cliente gestiona:

- Apertura y cierre del socket TCP.
- Autenticación con la contraseña RCON.
- Envío de comandos y lectura de respuestas.
- Gestión de timeouts para evitar bloqueos indefinidos.

Este servicio permite al sistema ejecutar cualquier comando de consola del servidor de CS:GO de forma remota y segura.

4.6 Servicio de procesamiento de logs públicos

`PublicMatchResultProcessor` analiza los ficheros de log generados por el servidor de CS:GO al final de cada partida. El proceso es:

1. Se lee el fichero de log del servidor.
2. Se busca mediante expresión regular la línea que indica el fin de la partida (cuando un equipo alcanza el número de rondas ganadas suficientes para ganar).
3. Se determina si es una partida 5v5 (umbral de 16 rondas) o Wingman (umbral de 9 rondas) basándose en el puerto del servidor.
4. Se extraen los SteamIDs de los jugadores de cada equipo.
5. Se aplican puntos a los ganadores y penalizaciones a los perdedores.
6. Se registra el resultado en la tabla `match\_results` con un hash de deduplicación.

7. Se utiliza un lock de caché para evitar que dos procesos simultáneos procesen el mismo log.

4.7 Panel de administración

El panel de administración es una vista única con acceso a todas las operaciones críticas:

- **KPIs del sistema**: Número total de usuarios, lobbies activos, servidores configurados y partidas jugadas.
- **Gestión de servidores**: Tabla con edición inline mediante formularios modales. CRUD completo de servidores de juego.
- **Gestión de usuarios**: Tabla con todos los usuarios registrados, con acciones para cambiar rol, banear/desbanear y modificar puntos y créditos directamente.
- **Limpieza operativa**: Botón para vaciar todos los lobbies de un tipo específico de servidor. Útil para reiniciar el estado del sistema sin acceder a la base de datos.

El acceso al panel está protegido por middleware que verifica el rol de administrador. Si un usuario no administrador intenta acceder a rutas `/admin/*`, es redirigido automáticamente.

4.8 Sistema de internacionalización

Todas las cadenas visibles de la interfaz están externalizadas en ficheros JSON en `lang/en.json` y `lang/es.json`. El sistema carga el idioma desde la sesión del usuario, que puede cambiar en cualquier momento desde el selector en la barra de navegación.

El middleware `SetLocale` se ejecuta en cada petición y establece el locale de Laravel (`App::setLocale()`), de modo que las funciones `__('clave')` resuelven automáticamente al idioma activo.

El sistema cubre 142 claves de traducción, incluyendo textos de navegación, estados del lobby, mensajes del sistema, etiquetas del panel administrativo y textos de la tienda.

5. Validación

5.1 Validación de flujos funcionales

Se han validado los siguientes flujos de forma manual y sistemática:

****Flujo de autenticación****

- El usuario accede a la página de bienvenida sin estar autenticado.
- Al hacer clic en "Iniciar sesión", es redirigido a Steam.
- Tras autenticarse en Steam, vuelve a la aplicación como usuario registrado.
- El perfil muestra el avatar y nombre de Steam correctamente.
- El cierre de sesión elimina la sesión y redirige a la página de bienvenida.

****Flujo de lobby (matchmaking)****

- Un usuario autenticado accede a la lista de servidores.
- Selecciona un servidor de tipo matchmaking y entra al lobby.
- Puede ver los jugadores presentes y sus estados de preparación.
- Puede unirse a un equipo (CT o T) y cambiar su estado de preparación.
- Si cierra el navegador sin salir, el sistema lo elimina del lobby en el siguiente ciclo de poda.
- Cuando todos los jugadores requeridos están listos, el sistema inicia el match.

****Flujo de administración****

- Un administrador accede al panel de admin.
- Puede crear un servidor con todos sus campos y verificar que aparece en la lista.
- Puede editar y eliminar servidores existentes.
- Puede cambiar el rol de un usuario, banearlo y desbanear.
- Puede ajustar puntos y créditos de cualquier usuario.
- La limpieza de lobbies vacía correctamente los registros correspondientes.

****Flujo de usuario baneado****

- Un usuario baneado que intenta acceder a rutas protegidas es redirigido a la página `/banned`.

- No puede acceder a servidores, lobbies ni otras funcionalidades.
- Solo puede cerrar sesión.

5.2 Validación de la integridad de datos

- Los `steam_id` son únicos en la tabla `users`. Si un usuario existente vuelve a autenticarse, se actualiza su registro en lugar de crear uno nuevo.
- Los registros de `lobby_user` tienen una restricción de unicidad sobre `[lobby_id, user_id]`, impidiendo que un usuario esté en el mismo lobby dos veces.
- El campo `event_hash` en `match_results` es único, garantizando que un mismo evento de Get5 no se procese más de una vez aunque el webhook lo envíe repetidamente.

5.3 Validación del entorno de instalación

La instalación del sistema en un servidor limpio fue validada siguiendo el procedimiento documentado en `README.md`:

1. Instalación de dependencias del sistema (`apt install`).
2. Clonado del repositorio.
3. `composer install` para dependencias PHP.
4. `npm install && npm run build` para el frontend.
5. Configuración del fichero `.env` con las variables necesarias.
6. `php artisan migrate --force` para crear el esquema de base de datos.
7. Configuración de Nginx y PHP-FPM.
8. Verificación de que la aplicación es accesible y funcional.

6. Pruebas

6.1 Estrategia de pruebas

El proyecto combina pruebas automatizadas con PHPUnit para los componentes de lógica pura y pruebas manuales para los flujos que requieren integraciones externas (Steam, Get5, RCON).

6.2 Pruebas unitarias automatizadas

Las pruebas unitarias se ubican en `tests/Unit/PublicMatchResultProcessorTest.php` y cubren el servicio de procesamiento de logs de servidores públicos, que es el componente con mayor lógica de negocio pura y menor dependencia de infraestructura externa.

Se ejecutan con:

php artisan test

o equivalentemente:

./vendor/bin/phpunit

Caso de prueba 1: Detección de victoria CT en partida 5v5

// Entrada: línea de log con Bomb_Defused y CT con 16 rondas

// Resultado esperado: winner_team = 'CT', ct_score = 16, t_score = 1

Verifica que el sistema detecta correctamente una victoria de CT cuando se alcanza el umbral de 16 rondas mediante Bomb_Defused.

Caso de prueba 2: Descarte de línea no terminal

// Entrada: línea con CTs_Win pero CT con solo 15 rondas

// Resultado esperado: null (no es final de partida)

Verifica que el sistema no procesa líneas de log de rondas ganadas intermedias, solo el final de partida.

Caso de prueba 3: Wingman — victoria CT a 9 rondas

// Entrada: Bomb_Defused, CT con 9 rondas, umbral = 9

// Resultado esperado: winner_team = 'CT', ct_score = 9, t_score = 3

Verifica el soporte para el modo Wingman (2v2) donde el umbral de victoria es 9 rondas en lugar de 16.

Caso de prueba 4: Wingman — victoria T a 9 rondas

// Entrada: Target_Bombed, T con 9 rondas, umbral = 9

// Resultado esperado: winner_team = 'TERRORIST', ct_score = 7, t_score = 9

Verifica la detección correcta del equipo Terrorista como ganador en modo Wingman.

Caso de prueba 5: Descarte de línea no terminal en Wingman

// Entrada: Target_Bombed pero marcador 8-8 (ningún equipo alcanza 9)

// Resultado esperado: null

Verifica que el sistema no produce falsos positivos cuando el marcador es empate y ningún equipo ha ganado.

Caso de prueba 6: Resolución del umbral según puerto del servidor

// Entrada: ruta de log conteniendo el puerto 27016 (Wingman)

// Resultado esperado: umbral = 9

Verifica que el sistema determina automáticamente si una partida es Wingman basándose en el puerto del servidor reflejado en el nombre del fichero de log, sin requerir configuración manual.

7.3 Cobertura de los casos de prueba

Caso	Método	Estado
------	--------	--------

#####

Victoria CT 5v5 (Bomb Defused)

<code>`test_extract_finished_match_supports_bomb_defused_results_for_public_5v5`</code>	Pasa
---	------

No procesar ronda no terminal

<code>`test_extract_finished_match_ignores_non_terminal_round_wins`</code>	Pasa
--	------

Victoria CT Wingman (9 rondas)

<code>`test_extract_finished_match_supports_wingman_ct_win_at_nine_rounds`</code>	Pasa
---	------

Victoria T Wingman (9 rondas)

<code>`test_extract_finished_match_supports_wingman_t_win_at_nine_rounds`</code>	Pasa
--	------

Descarte no terminal Wingman

<code>`test_extract_finished_match_ignores_wingman_non_terminal_loss_lines`</code>	Pasa
--	------

Resolución umbral por puerto

<code>`test_resolve_winning_score_uses_wingman_threshold_for_27016_logs`</code>	Pasa
---	------

6.4 Pruebas manuales realizadas

Las siguientes pruebas se realizaron manualmente en el entorno de desarrollo:

Funcionalidad	Acción	Resultado esperado	Verificado
---------------	--------	--------------------	------------

#####

Autenticación	Login con cuenta Steam real	Usuario creado/actualizado en BD	Sí
---------------	-----------------------------	----------------------------------	----

Autenticación	Cierre de sesión	Sesión destruida, redirección a inicio	Sí
---------------	------------------	--	----

Lobby	Entrar a lobby	Usuario aparece en la lista	Sí
-------	----------------	-----------------------------	----

Lobby	Seleccionar equipo CT	Usuario asignado a CT	Sí
-------	-----------------------	-----------------------	----

| Lobby | Marcar como listo | Estado de preparación actualizado | Sí |

| Lobby | Cerrar navegador | Usuario eliminado tras 20s | Sí |

| Admin | Crear servidor | Servidor aparece en lista | Sí |

| Admin | Banear usuario | Usuario redirigido a `/banned` | Sí |

| Admin | Modificar puntos | Puntos actualizados en perfil | Sí |

| Admin | Limpiar lobbies | Lobbies vaciados | Sí |

| Ranking | Ver ranking | Top 50 por puntos mostrado | Sí |

| Idioma | Cambiar a inglés | Interfaz en inglés | Sí |

| Idioma | Cambiar a español | Interfaz en español | Sí |

6.5 Pruebas de integración con Get5 (manual)

La integración con Get5 fue validada mediante el envío manual de payloads JSON al endpoint del webhook usando herramientas de petición HTTP:

- Se simuló un evento `series\_result` con equipos ficticios.
- Se verificó que el sistema registró el resultado en `match\_results`.
- Se verificó que los puntos se aplicaron correctamente a los jugadores.
- Se verificó la protección contra duplicados enviando el mismo payload dos veces.

7. Resultados

7.1 Estado del sistema al finalizar el proyecto

AfterReload es una plataforma web funcional y desplegada que resuelve de forma efectiva los objetivos planteados en la fase de análisis. El sistema está en uso activo por la comunidad para la que fue diseñado.

****Funcionalidades implementadas y operativas:****

- Autenticación completa mediante Steam OAuth.
- Gestión de servidores con diferenciación de tipos y estados.
- Sistema de lobbies con presencia activa, selección de equipo y estado de preparación.
- Integración con Get5 para el ciclo completo de partidas competitivas.
- Procesamiento automático de resultados en servidores públicos.
- Sistema de puntos y créditos para la progresión del jugador.
- Ranking público de los 50 mejores jugadores.
- Panel de administración centralizado.
- Interfaz completamente traducida al español e inglés.
- 6 pruebas unitarias automatizadas con PHPUnit, todas en estado pasado.

7.2 Métricas del proyecto

| Métrica | Valor |

#####

| Líneas de código PHP (app/) | ~1.300 líneas |

| Controladores | 7 |

| Servicios | 3 |

| Modelos Eloquent | 5 |

| Migraciones | 10 |

| Rutas definidas | 20 |

| Vistas Blade | 12 |

| Claves de traducción | 142 |

| Pruebas automatizadas | 6 |

| Tablas en base de datos | 11 |

7.3 Logros destacados

****Reducción de dependencias externas**:** El sistema centraliza la gestión que antes requería múltiples herramientas. Los administradores de la comunidad no necesitan acceder a paneles de terceros para las operaciones habituales.

****Consistencia de lobbies**:** La introducción del sistema de presencia con TTL de 20 segundos resolvió el problema crónico de los jugadores fantasma, que anteriormente requería intervención manual para limpiar los lobbies.

****Deduplicación de eventos**:** El mecanismo de hash SHA-256 garantiza que ningún resultado de partida se procese más de una vez, incluso si el servidor de juego reenvía el mismo evento varias veces.

****Integración RCON nativa**:** La implementación del protocolo RCON en PHP puro elimina la necesidad de herramientas externas para el control remoto de servidores de juego.

****Soporte multipartida**:** El sistema diferencia correctamente entre partidas 5v5 estándar y Wingman 2v2, aplicando umbrales de victoria distintos según el modo de juego.

7.4 Limitaciones identificadas

- ****Tiempo real real**:** La actualización de la vista de servidores y lobbies se realiza mediante polling (peticiones periódicas). Una implementación con WebSockets o Server-Sent Events ofrecería una experiencia más fluida, pero requeriría infraestructura adicional (servidor de broadcasting como Pusher o Soketi).
- ****Estadísticas avanzadas**:** El sistema registra puntos y créditos, pero no estadísticas detalladas por partida (kills, muertes, asistencias, headshots). Esto requeriría procesar más información de los logs de Get5.
- ****Observabilidad**:** No existe un panel de monitorización de la aplicación. Los errores se registran en los logs de Laravel, pero no hay alertas automáticas ni métricas de rendimiento.

- **Cobertura de pruebas**: Las pruebas automatizadas cubren únicamente el servicio de procesamiento de logs. Los controladores y la lógica de lobbies no tienen pruebas automatizadas, dependiendo de la verificación manual.

7.5 Líneas de trabajo futuras

1. **WebSockets para actualización en tiempo real**: Implementar Laravel Echo con Pusher o Soketi para reemplazar el polling por actualizaciones push.
2. **Estadísticas detalladas por partida**: Procesar los logs de Get5 para extraer estadísticas individuales por jugador y partida.
3. **Sistema de notificaciones**: Alertas al usuario cuando su lobby está a punto de comenzar o cuando recibe una invitación.
4. **Integración con mapas**: Permitir a los administradores especificar el mapa para cada match desde el panel, en lugar de usar siempre el mapa por defecto.
5. **API pública documentada**: Exponer una API REST documentada con OpenAPI/Swagger para que herramientas externas puedan consultar estadísticas y estado del sistema.
6. **Panel de observabilidad**: Dashboard de métricas de la aplicación con Grafana o una solución equivalente.
7. **Cobertura de pruebas**: Añadir pruebas de integración y de funcionalidad para los controladores principales, especialmente `LobbyController` y `Get5Controller`.